

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	C. Sandeep	Reg. No.:	192472368
Section / Batch:	AI / 2024	Date:	

Code of Conduct

I **C.Sandeep (192472368)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C.Sandeep

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Porter Stemmer	class design, methods, encapsulation, and rule-based stemming	1

Annexure A – Coding Challenge

Section 1: Object-Oriented Implementation of a Porter Stemmer

You have recently joined an **NLP startup** called **LexiMind Technologies**. The company is developing a **search engine** for a digital library containing millions of research papers. Users expect the search engine to return relevant results even when different grammatical forms of a word are used.

For example, a search for **connect** should also retrieve documents containing **connected**, **connecting**, **connection**, and **connections**. To achieve this, the development team has decided to implement the **Porter Stemming Algorithm** from scratch.

Since the algorithm consists of multiple sequential rules (Step 1a, Step 1b, Step 1c, Step 2, Step 3, Step 4, Step 5a, and Step 5b), the project manager has divided the work among team members. Each developer is responsible for implementing and testing one specific rule. At the end of the project, all modules must be integrated into a single, fully functional stemmer.

Assessment Task

Phase 1: Individual Task (Assigned Rule)

Each student will be assigned **one Porter Stemmer rule**.

Your responsibilities are to:

1. Study and understand the assigned rule.
2. Explain the linguistic purpose of the rule.
3. Describe the conditions under which the rule should be applied.
4. Implement the rule in Python.
5. Prepare at least **10 test words** demonstrating:
 - Words that satisfy the rule.
 - Words that should remain unchanged.
6. Document the input, expected output, and actual output.
7. Explain any limitations or special cases encountered.

Phase 2: Team Integration

As a team, integrate all individual rule implementations into a complete Porter Stemmer.

The team must:

- Arrange the rules in the correct execution order.
- Resolve conflicts between rule implementations.
- Ensure that the output of one rule becomes the input to the next rule.
- Test the complete algorithm using at least **50 different English words**.
- Compare the team's results with the NLTK Porter Stemmer.
- Identify and explain any differences.

Phase 3: Reflection

Prepare a report addressing the following questions:

1. Why is the order of Porter Stemmer rules important?
2. What errors would occur if your assigned rule were skipped?
3. How did integrating multiple rules improve stemming accuracy?
4. Which rule was the most challenging to implement and why?
5. What improvements would you suggest for the Porter Stemming algorithm?

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach

Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1	15	18	15	20	12		80

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: Kumaragurubaran T	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Step 4 of the Porter Stemming Algorithm

Aim

To implement **Step 4** of the Porter Stemming Algorithm in Python and understand how suffix removal reduces words to their root form while preserving their semantic meaning.

Introduction

Stemming is one of the most important preprocessing techniques in **Natural Language Processing (NLP)**. It converts different forms of a word into a common base or stem. This process reduces the vocabulary size and improves the efficiency of tasks such as text classification, information retrieval, sentiment analysis, and search engines.

The **Porter Stemming Algorithm**, developed by Martin Porter (1980), is a rule-based stemming algorithm consisting of five major steps. Among these, **Step 4** is responsible for removing common derivational suffixes from words after ensuring that the remaining stem is linguistically meaningful.

Objective of Step 4

The objective of Step 4 is:

- To remove unnecessary suffixes.
- To shorten words to their meaningful stems.
- To reduce variations of the same word.
- To ensure that suffix removal does not over-stem the word.

Python Code for Step 4

main.py



Share

Run

```
1 def is_vowel(word, index):
2     ch = word[index].lower()
3
4     if ch in "aeiou":
5         return True
6
7     if ch == 'y':
8         if index == 0:
9             return False
10        return not is_vowel(word, index - 1)
11
12    return False
13 def measure(stem):
14     m = 0
15     prev_vowel = False
16
17     for i in range(len(stem)):
18         if is_vowel(stem, i):
19             prev_vowel = True
20         else:
21             if prev_vowel:
22                 m += 1
23             prev_vowel = False
24
25     return m
```

main.py



Share

Run

```
26
27 def step4(word):
28
29     suffixes = [
30         "al", "ance", "ence", "er",
31         "ic", "able", "ible",
32         "ant", "ement", "ment",
33         "ent", "ion", "ou",
34         "ism", "ate", "iti",
35         "ous", "ive", "ize"
36     ]
37
38     for suffix in suffixes:
39
40         if word.endswith(suffix):
41
42             stem = word[:-len(suffix)]
43
44             if suffix == "ion":
45                 if measure(stem) > 1 and (
46                     stem.endswith("s") or stem.endswith("t")):
47                     return stem
48
49             else:
50                 if measure(stem) > 1:
```

```

52
53     return word
54
55
56 word = input("Enter a word: ")
57
58 print("Original Word :", word)
59 print("Stemmed Word :", step4(word))

```

Output

Clear

```

Enter a word: introduction
Original Word : introduction
Stemmed Word : introduct

=== Code Execution Successful ===

```

Output

```

Enter a word: allowance
Original Word : allowance
Stemmed Word : allow

=== Code Execution Successful ===

```

Code Explanation

Function Definition

```
def step4(word):
```

This function accepts a word after it has completed Steps 1, 2, and 3 of the Porter algorithm.

Example:

revival

allowance

adjustment

List of Suffixes

```

suffixes = [
    "al", "ance", "ence", "er", "ic",
    "able", "ible", "ant", "ement",
    "ment", "ent", "ion", "ou",
    "ism", "ate", "iti", "ous",
    "ive", "ize"
]

```

These are the suffixes that Porter Step 4 attempts to remove.

Examples

Word	Suffix
revival	al
allowance	ance
electrical	ic
adjustment	ment
activity	iti
formalize	ize

Loop Through Every Suffix

for suffix in suffixes:

The program checks each suffix one by one.

Suppose the word is

adjustment

It checks

al

ance

ence

...

ment

When it reaches **ment**, the condition becomes true.

Check Whether the Word Ends with the Suffix

if word.endswith(suffix):

Python's **endswith()** function checks whether the word ends with the current suffix.

Example

adjustment.endswith("ment")

True

Extract the Stem

stem = word[:-len(suffix)]

The suffix is removed.

Example

adjustment

↓

adjust

Similarly,

allowance

↓

allow

Measure (m) Condition

Before removing a suffix, Porter checks

$\text{measure}(\text{stem}) > 1$

The **measure (m)** counts the number of **Vowel-Consonant (VC)** sequences in the stem.

Examples

Stem	Measure
hop	1
trouble	2
revival	3

If

$m > 1$

then suffix removal is allowed.

Otherwise, the word remains unchanged.

Special Rule for "ion"

Unlike every other suffix,

ion

has an additional condition.

if `suffix=="ion"`:

The algorithm checks

`measure(stem)>1`

AND

`stem.endswith("s")`

OR

`stem.endswith("t")`

Only then is **ion** removed.

General Condition of Step 4

The algorithm follows the rule

IF

Word ends with one of the Step 4 suffixes

AND

$\text{measure}(\text{stem}) > 1$

THEN

Remove the suffix

For

ion

the condition becomes

Word ends with "ion"

AND

$\text{measure}(\text{stem}) > 1$

AND

Stem ends with

s

OR

t

THEN

Remove "ion"

Conclusion

Step 4 is one of the most important stages of the Porter Stemming Algorithm because it removes common derivational suffixes only when the remaining stem is linguistically meaningful. By applying the **measure (m) > 1** condition and the special **"ion"** rule, the algorithm avoids excessive stemming while effectively reducing words to their base forms. This normalization improves the performance of NLP tasks such as information retrieval, text mining, document classification, and search engines by treating different word forms as a single representative stem.